

Introduction to Advanced Bash Scripting

Estimated time needed: **5 minutes**

In the hands-on lab portion of the final project, you will be using more advanced scripting commands and concepts that the course has not covered yet. This reading will familiarize you with these more advanced concepts, so you can complete the lab with confidence.

Objectives

After completing this reading, you will be able to create Bash scripts that:

- use conditional statements to run a set of commands only if a specified condition is `true`
- apply logical operators to create `true` / `false` comparisons
- perform basic arithmetic calculations
- create list-like arrays and access their elements
- implement `for` loops to execute operations repeatedly, based on a looping index

Conditionals

Conditionals, or `if` statements, are a way of telling a script to do something only under a specific condition.

Bash script conditionals use the following `if` - `then` - `else` syntax:



bash



```
if [ condition ]
then
    statement_block_1
else
    statement_block_2
fi
```

If the **condition** is **true**, then Bash executes the statements in **statement_block_1** before exiting the conditional block of code. After exiting, it will continue to run any commands after the closing **fi**.

Alternatively, if the **condition** is **false**, Bash instead runs the statements in **statement_block_2** under the **else** line, then exits the conditional block and continues to run commands after the closing **fi**.

Tips:

- You must always put spaces around your condition within the square brackets **[]**.
- Every **if** condition block must be paired with a **fi** to tell Bash where the condition block ends.
- The **else** block is optional but recommended. If the condition evaluates to **false** without an **else** block, then nothing happens within the **if** condition block. Consider options such as echoing a comment in **statement_block_2** to indicate that the condition was evaluated as **false**.

In the following example, the condition is checking whether the number of command-line arguments read by some Bash script, **\$#**, is equal to 2.



php



```
if [[ $# == 2 ]]
then
    echo "number of arguments is equal to 2"
else
    echo "number of arguments is not equal to 2"
fi
```

Both single [] and double [[]] brackets support integer comparisons when using operators.

You can also make string comparisons. For example, assume you have a variable called `string_var` that has the value `"Yes"` assigned to it. Then the following statement evaluates to `true`:



php



```
`[ $string_var == "Yes" ]`
```

Notice you only need single square brackets when making string comparisons.

You can also include multiple conditions to be satisfied by using the "and" operator `&&` or the "or" operator `||`. For example:



php



```
if [ condition1 ] && [ condition2 ]
then
    echo "conditions 1 and 2 are both true"
else
    echo "one or both conditions are false"
fi
```



php



```
if [ condition1 ] || [ condition2 ]
then
    echo "conditions 1 or 2 are true"
else
    echo "both conditions are false"
fi
```

Logical operators

The following logical operators can be used to compare integers within a condition in an **if** condition block.

==: is equal to

If a variable **a** has a value of 2, the following condition evaluates to **true**; otherwise it evaluates to **false**.



php



```
$a == 2
```

!=: is not equal to

If a variable **a** has a value different from 2, the following statement evaluates to **true**. If its value is 2, then it evaluates to **false**.



plaintext



```
a != 2
```

*Tip: The **!** logical negation operator changes **true** to **false** and **false** to **true**.*

<=: is less than or equal to

If a variable **a** has a value of 2, then the following statement



plaintext



```
a <= 3
```

and the following statement evaluates to **false** :



plaintext



```
a <= 1
```

Alternatively, you can use the equivalent notation **-le** in place of **<=** :



php



```
a=1
b=2
if [ $a -le $b ]
then
    echo "a is less than or equal to b"
else
    echo "a is not less than or equal to b"
fi
```

We've only provided a small sampling of logical operators here. You can explore resources such as the [Advanced Bash-Scripting Guide](#) to find out more.

Arithmetic calculations

You can perform integer addition, subtraction, multiplication, and division using the notation **\$(())**. For example, the following two sets of commands both display the result of adding 3 and 2.



php



```
echo $((3+2))
```

php

```
a=3
b=2
c=$(( $a+$b ))
echo $c
```

Bash natively handles integer arithmetic but does not handle floating-point arithmetic. As a result, it will always truncate the decimal portion of a calculation result.

For example:

php

```
echo $((3/2))
```

prints the truncated integer result, **1**, not the floating-point number, **1.5**.

The following table summarizes the basic arithmetic operators:

Symbol	Operation
+	addition
-	subtraction
*	multiplication
/	division

Table: Arithmetic operators

Arrays

The **array** is a Bash built-in data structure. An array is a space-delimited list contained in parentheses. To create an array, declare its name and contents:



plaintext



```
my_array=(1 2 "three" "four" 5)
```

This statement creates and populates the array `my_array` with the items in the parentheses: `1`, `2`, `"three"`, `"four"`, and `5`.

You can also create an empty array by using:



plaintext



```
declare -a empty_array
```

If you want to add items to your array after creating it, you can add to your array by appending one element at a time:



plaintext



```
my_array+=("six")  
my_array+=(7)
```

This adds elements `"six"` and `7` to the array `my_array`.

By using indexing, you can access individual or multiple elements of an array:



php



```
# print the first item of the array:  
echo ${my_array[0]}  
  
# print the third item of the array:  
echo ${my_array[2]}  
  
# print all array elements:  
echo ${my_array[@]}
```

Tip: Note that array indexing starts from 0, not from 1.

for loops

You can use a construct called a **for** loop along with indexing to iterate over all elements of an array.

For example, the following **for** loops will continue to run over and over again until every element is printed:

 ruby 

```
for item in ${my_array[@]}; do
    echo $item
done
```

or

 ruby 

```
for i in ${!my_array[@]}; do
    echo ${my_array[$i]}
done
```

The **for** loop requires a **; do** component in order to cycle through the loop. Additionally, you need to terminate the **for** loop block with a **done** statement.

Another way to implement a **for** loop when you know how many iterations you want is as follows. For example, the following code prints the number 0 through 6.

 ruby 

```
N=6
for (( i=0; i<=$N; i++ )) ; do
    echo $i
done
```

You can use **for** loops to accomplish all sorts of things. For example, you could count the number of items in an array or sum up its elements, as the following Bash script does:



ruby



```
#!/usr/bin/env bash
# initialize array, count, and sum
my_array=(1 2 3)
count=0
sum=0
for i in ${!my_array[@]}; do
    # print the ith array element
    echo ${my_array[$i]}
    # increment the count by one
    count=$((count+1))
    # add the current value of the array to the sum
    sum=$((sum+${my_array[$i]}))
done
echo $count
echo $sum
```

Go ahead and try running this script, so you get a sense of how this loop works.

Summary

In this lab, you learned that:

- Conditional statements can be used to run commands based on whether a specified condition is **true**
- Logical operators do **true** / **false** comparisons
- Arithmetic operators perform basic arithmetic calculations
- You can create list-like arrays and access their individual elements
- **for** loops execute operations repeatedly, based on a looping index

Congratulations! You are now ready to practice your newly acquired knowledge in the following hands-on lab.

Authors

Introduction to Advanced Bash Scripting

Estimated time needed: **5 minutes**

In the hands-on lab portion of the final project, you will be using more advanced scripting commands and concepts that the course has not covered yet. This reading will familiarize you with these more advanced concepts, so you can complete the lab with confidence.

Objectives

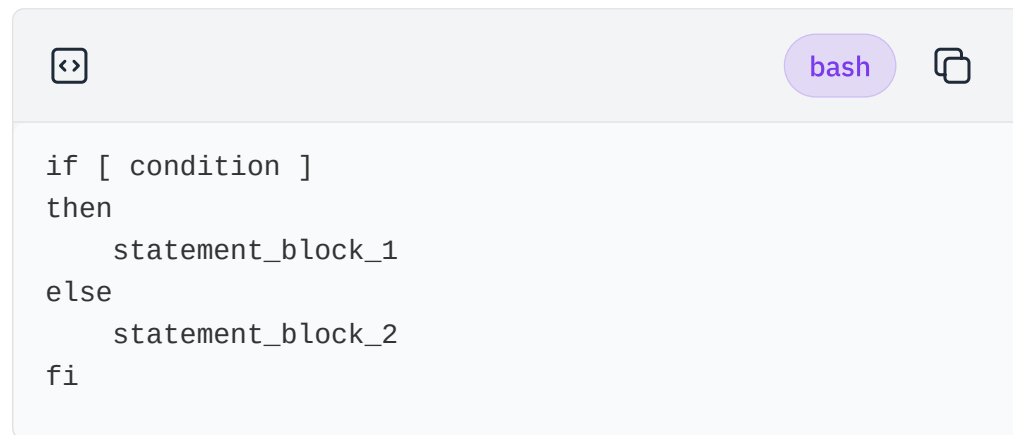
After completing this reading, you will be able to create Bash scripts that:

- use conditional statements to run a set of commands only if a specified condition is `true`
- apply logical operators to create `true` / `false` comparisons
- perform basic arithmetic calculations
- create list-like arrays and access their elements
- implement `for` loops to execute operations repeatedly, based on a looping index

Conditionals

something only under a specific condition.

Bash script conditionals use the following **if** - **then** - **else** syntax:

A code editor window with a light gray background. At the top, there is a toolbar with a code icon (two arrows pointing out of a box) on the left, a purple pill-shaped button with the text 'bash' in the center, and a copy icon (two overlapping squares) on the right. Below the toolbar, the code is displayed in a monospaced font. The code is an if-then-else block: 'if [condition]' followed by 'then' on the next line, then 'statement_block_1' indented. This is followed by 'else' on the next line, then 'statement_block_2' indented. The block ends with 'fi' on the final line.

```
if [ condition ]
then
    statement_block_1
else
    statement_block_2
fi
```

If the **condition** is **true**, then Bash executes the statements in **statement_block_1** before exiting the conditional block of code. After exiting, it will continue to run any commands after the closing **fi**.

Alternatively, if the **condition** is **false**, Bash instead runs the statements in **statement_block_2** under the **else** line, then exits the conditional block and continues to run commands after the closing **fi**.

Tips:

- *You must always put spaces around your condition within the square brackets **[]**.*
- *Every **if** condition block must be paired with a **fi** to tell Bash where the condition block ends.*
- *The **else** block is optional but recommended. If the condition evaluates to **false** without an **else** block, then nothing happens within the **if** condition block. Consider options such as echoing a comment in **statement_block_2** to indicate that the condition was evaluated as **false**.*

In the following example, the condition is checking whether the number of command-line arguments read by some Bash script, **\$#**, is equal to 2.



php



```
if [[ $# == 2 ]]
then
    echo "number of arguments is equal to 2"
else
    echo "number of arguments is not equal to 2"
fi
```

Both single [] and double [[]] brackets support integer comparisons when using operators.

You can also make string comparisons. For example, assume you have a variable called `string_var` that has the value `"Yes"` assigned to it. Then the following statement evaluates to `true`:



php



```
`[ $string_var == "Yes" ]`
```

Notice you only need single square brackets when making string comparisons.

You can also include multiple conditions to be satisfied by using the "and" operator `&&` or the "or" operator `||`. For example:



php



```
if [ condition1 ] && [ condition2 ]
then
    echo "conditions 1 and 2 are both true"
else
    echo "one or both conditions are false"
fi
```



php



```
if [ condition1 ] || [ condition2 ]
then
    echo "conditions 1 or 2 are true"
else
    echo "both conditions are false"
fi
```

Logical operators

The following logical operators can be used to compare integers within a condition in an **if** condition block.

==: is equal to

If a variable **a** has a value of 2, the following condition evaluates to **true**; otherwise it evaluates to **false**.



php



```
$a == 2
```

!=: is not equal to

If a variable **a** has a value different from 2, the following statement evaluates to **true**. If its value is 2, then it evaluates to **false**.



plaintext



```
a != 2
```

*Tip: The **!** logical negation operator changes **true** to **false** and **false** to **true**.*

<=: is less than or equal to

If a variable **a** has a value of 2, then the following statement



plaintext



```
a <= 3
```

and the following statement evaluates to **false** :



plaintext



```
a <= 1
```

Alternatively, you can use the equivalent notation **-le** in place of **<=** :



php



```
a=1
b=2
if [ $a -le $b ]
then
    echo "a is less than or equal to b"
else
    echo "a is not less than or equal to b"
fi
```

We've only provided a small sampling of logical operators here. You can explore resources such as the [Advanced Bash-Scripting Guide](#) to find out more.

Arithmetic calculations

You can perform integer addition, subtraction, multiplication, and division using the notation **\$(())**. For example, the following two sets of commands both display the result of adding 3 and 2.



php



```
echo $((3+2))
```

php

```
a=3
b=2
c=$(( $a+$b ))
echo $c
```

Bash natively handles integer arithmetic but does not handle floating-point arithmetic. As a result, it will always truncate the decimal portion of a calculation result.

For example:

php

```
echo $((3/2))
```

prints the truncated integer result, **1**, not the floating-point number, **1.5**.

The following table summarizes the basic arithmetic operators:

Symbol	Operation
+	addition
-	subtraction
*	multiplication
/	division

Table: Arithmetic operators

Arrays

The **array** is a Bash built-in data structure. An array is a space-delimited list contained in parentheses. To create an array, declare its name and contents:



plaintext



```
my_array=(1 2 "three" "four" 5)
```

This statement creates and populates the array `my_array` with the items in the parentheses: `1`, `2`, `"three"`, `"four"`, and `5`.

You can also create an empty array by using:



plaintext



```
declare -a empty_array
```

If you want to add items to your array after creating it, you can add to your array by appending one element at a time:



plaintext



```
my_array+=("six")  
my_array+=(7)
```

This adds elements `"six"` and `7` to the array `my_array`.

By using indexing, you can access individual or multiple elements of an array:



php



```
# print the first item of the array:  
echo ${my_array[0]}  
  
# print the third item of the array:  
echo ${my_array[2]}  
  
# print all array elements:  
echo ${my_array[@]}
```

Tip: Note that array indexing starts from 0, not from 1.

for loops

You can use a construct called a **for** loop along with indexing to iterate over all elements of an array.

For example, the following **for** loops will continue to run over and over again until every element is printed:

 ruby 

```
for item in ${my_array[@]}; do
    echo $item
done
```

or

 ruby 

```
for i in ${!my_array[@]}; do
    echo ${my_array[$i]}
done
```

The **for** loop requires a **; do** component in order to cycle through the loop. Additionally, you need to terminate the **for** loop block with a **done** statement.

Another way to implement a **for** loop when you know how many iterations you want is as follows. For example, the following code prints the number 0 through 6.

 ruby 

```
N=6
for (( i=0; i<=$N; i++ )) ; do
    echo $i
done
```

You can use **for** loops to accomplish all sorts of things. For example, you could count the number of items in an array or sum up its elements, as the following Bash script does:



ruby



```
#!/usr/bin/env bash
# initialize array, count, and sum
my_array=(1 2 3)
count=0
sum=0
for i in ${!my_array[@]}; do
    # print the ith array element
    echo ${my_array[$i]}
    # increment the count by one
    count=$((count+1))
    # add the current value of the array to the sum
    sum=$((sum+${my_array[$i]}))
done
echo $count
echo $sum
```

Go ahead and try running this script, so you get a sense of how this loop works.

Summary

In this lab, you learned that:

- Conditional statements can be used to run commands based on whether a specified condition is **true**
- Logical operators do **true** / **false** comparisons
- Arithmetic operators perform basic arithmetic calculations
- You can create list-like arrays and access their individual elements
- **for** loops execute operations repeatedly, based on a looping index

Congratulations! You are now ready to practice your newly acquired knowledge in the following hands-on lab.

Authors

Other Contributors

Rav Ahuja



Skills Network

